Low-Rank Approximation with the SVD

Contents

1. Overview 2
2. A matrix as a sum of rank-1 layers 3
   - 2.1 A table that is a column times a row 3
   - 2.2 Splitting any matrix into layers 5
3. The rank k approximation 9
4. The best approximation: Eckart–Young 11
   - 4.1 Measuring the size of a matrix 11
   - 4.2 The Eckart–Young theorem 13
5. Compressing an image 16
6. How many singular values to keep 18
7. Removing noise 19
8. Summary 22
9. Sources 22
10. Key terms 23

Where this fits: Pipeline map step 6 (Reduce dimensions). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	**6 Reduce**
7 Split	8 Model	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Singular value decomposition.
- **Compare with:** PCA.

1. Overview

Key point: Many tables of numbers are mostly one simple pattern plus small corrections. The SVD splits any matrix into layers, biggest first. Keeping only the big layers and dropping the small ones gives the best possible small summary of the matrix, and the size of its error is the biggest layer we dropped.

A small shop keeps a table of how many boxes of ice cream each of its three branches sells on each of four days. The large branch always sells about three times what the small branch sells, and Sunday is always about three times as busy as Monday. So the whole table is almost "branch size times day busyness": one simple pattern, plus a few small surprises. Writing down the pattern takes 7 numbers instead of 12. Keeping the pattern and dropping the surprises is what this Note calls a low-rank approximation.

A photo works the same way, only bigger.

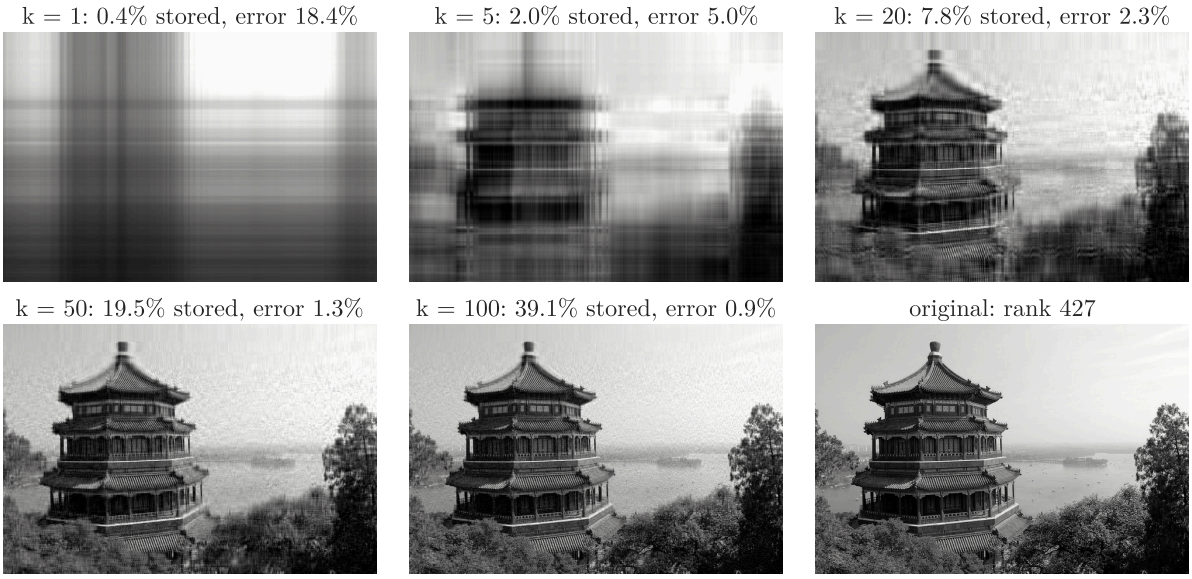


Figure 1: A 427×640 photo rebuilt from its first k layers: "stored" is the share of numbers kept, "error" is the size of the biggest dropped layer relative to the biggest kept one

Figure 1 shows the payoff. Each panel is the same photo rebuilt from its first k layers: read the panels left to right, top row first, with k growing from 1 to 100, and the last panel is the

original. The grey photo is a table of 273,280 brightness values. Built from its first k layers, it is recognisable from 8 percent of the numbers ($k = 20$), and close to the original from 20 percent ($k = 50$).

The SVD (singular value decomposition) writes a matrix (a table of numbers) as a product of three factors U, Σ and V (rotate, stretch, rotate), and the recipe worked on a 2×2 matrix found them by hand. Now we use them. The formal grounding is *Mathematics for Machine Learning* (Deisenroth, Faisal, Ong, 2020; MML below), §4.6 Matrix Approximation.

This Note covers:

- a table that is one column times one row (Section 2.1), and any matrix as a sum of such layers (Section 2.2);
- keeping only the first k layers, the **rank** (G-1627) k approximation (Section 3);
- why it is the best one possible, the **Eckart–Young theorem** (G-657) (Section 4);
- compressing an image (Section 5) and choosing k (Section 6);
- removing noise (Section 7).

2. A matrix as a sum of rank-1 layers

> **Key point:** The simplest kind of table is one column of numbers times one row of numbers. The SVD writes every matrix as a sum of such tables, each scaled by a **singular value** (G-1812), biggest first.

2.1 A table that is a column times a row

> **Key point:** If every row of a table is a multiple of the same row, the whole table is one column times one row: an outer product, a matrix of rank 1.

The idea in plain words. Start with the shop's plan. Each branch has a size, and each day has a busyness:

Branch	Size
small	1
medium	2
large	3

Day	Mon	Tue	Sat	Sun
Busyness	2	2	4	6

The plan says: boxes sold = branch size $\times$ day busyness. Each entry of the table is one size times one busyness.

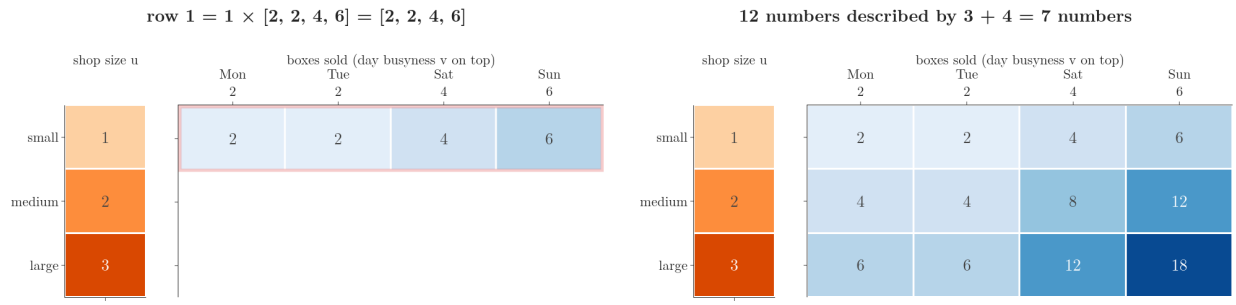


Figure 2: The plan table built row by row: each row is the branch size (orange, left) times the row of day busyness [2, 2, 4, 6] (top). The finished table has 12 entries but is fully described by $3 + 4 = 7$ numbers

Worked example, row by row. Figure 2 fills the table one row at a time. Watch the red box move down:

$$\text{small row} = 1 \times [2, 2, 4, 6] = [2, 2, 4, 6]$$

$$\text{medium row} = 2 \times [2, 2, 4, 6] = [4, 4, 8, 12]$$

$$\text{large row} = 3 \times [2, 2, 4, 6] = [6, 6, 12, 18]$$

Every row is a multiple of the same row $[2, 2, 4, 6]$, and every column is a multiple of the same column $[1, 2, 3]$. The table has 12 entries, but 7 numbers (3 sizes and 4 busyness values) describe it completely.

The standard terms. A column of numbers times a row of numbers is an **outer product** (G-1418). We write the column as $\mathbf{u}$ and the row as $\mathbf{v}^\mathsf{T}$, where the T (**transpose**, G-2012) turns the column $\mathbf{v}$ on its side into a row. Here

$$\mathbf{u} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

$$\mathbf{v}^\mathsf{T} = \begin{bmatrix} 2 & 2 & 4 & 6 \end{bmatrix}$$

The **rank** (G-1627) of a matrix counts how many different row patterns it really has (the number of dimensions its outputs fill; see **span** (G-1838)). Every row of an outer product is a multiple of one row, so an outer product has rank 1.

The formal version. A matrix with m rows and n columns is called $m \times n$: the plan table is 3×4. For a column $\mathbf{u}$ with m entries and a column $\mathbf{v}$ with n entries, the outer product $\mathbf{u}\mathbf{v}^\mathsf{T}$ is the $m \times n$ matrix whose entry in row i, column j is $u_i v_j$. With two entries each:

$$\mathbf{uv}^\mathsf{T} = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} \begin{bmatrix} v_1 & v_2 \end{bmatrix}$$

$$\mathbf{uv}^\mathsf{T} = \begin{bmatrix} u_1 v_1 & u_1 v_2 \\ u_2 v_1 & u_2 v_2 \end{bmatrix}$$

Check on the plan table: row 2, column 4 is

$$u_2 v_4 = 2 \times 6 = 12$$

as in Figure 2. A smaller example, used again below:

$$\begin{bmatrix} 1 \\ 3 \end{bmatrix} \begin{bmatrix} 1 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 1 \\ 3 & 3 \end{bmatrix}$$

An outer product needs only $m + n$ numbers to describe, instead of $m \times n$. The dot product does the multiplication the other way round, a row times a column, and gives one number (see the dot product as a matrix product).

2.2 Splitting any matrix into layers

> **Key point:** A real table is not exactly one **outer product** (G-1418). The SVD splits it into a big outer product plus smaller ones, each weighted by a singular value.

The idea in plain words. Real sales never follow the plan exactly. Suppose the large branch, which sits next to a park, sold one extra box on Saturday and on Sunday. Its row is now $[6, 6, 13, 19]$, and that row is no longer a multiple of $[2, 2, 4, 6]$. The table is no longer a single outer product. But it is still almost one: a big pattern plus a small correction.

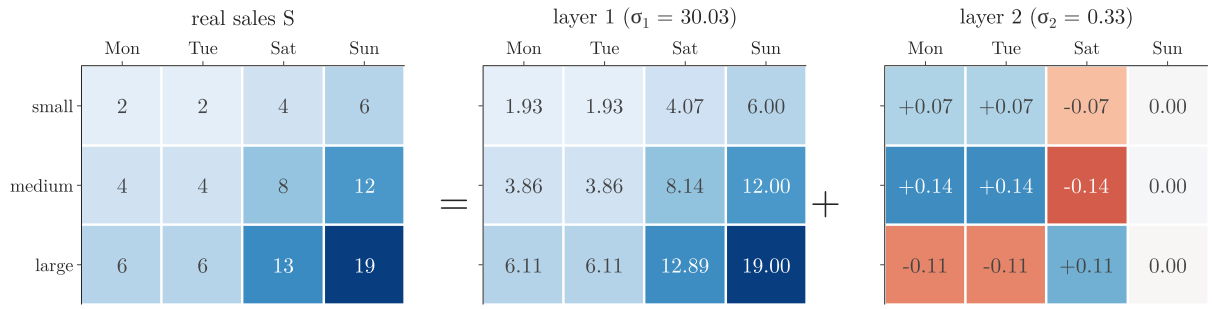


Figure 3: The real sales table S (left) split by the SVD into layer 1 (middle), a column times a row scaled by $\sigma_1 = 30.03$, plus layer 2 (right), a much smaller column times a row scaled by $\sigma_2 = 0.33$. Blue is positive, red negative

Figure 3 shows the split. Read it as a sum, entry by entry. For the large branch on Saturday:

$$\text{real sales} = 13$$

$$\text{layer 1} = 12.89$$

$$\text{layer 2} = +0.11$$

$$12.89 + 0.11 = 13.00$$

Layer 1 carries nearly everything: each of its entries is within 0.14 of the real table. Layer 2 is the small correction for the park.

The standard terms. Each piece is a **rank-1 layer** (G-1631). The weight of layer i is its **singular value** (G-1812), written σ_i (the Greek letter "sigma"): here $\sigma_1 = 30.03$ and $\sigma_2 = 0.33$. The column of layer i is the **left singular vector** (G-1079) $\mathbf{u}_i$, and its row is the transposed **right singular vector** (G-1692) $\mathbf{v}_i^{\mathsf{T}}$; both have length 1, so all the size sits in σ_i. The singular values come out sorted, biggest first.

Worked example on the 2 × 2 matrix. The SVD Notes used

$$A = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix}$$

The recipe worked on a 2 × 2 matrix found its factors:

i	σ_i	$\mathbf{u}_i$	$\mathbf{v}_i$
1	$3\sqrt{5} = 6.71$	$\frac{1}{\sqrt{10}}[1, 3]$	$\frac{1}{\sqrt{2}}[1, 1]$
2	$\sqrt{5} = 2.24$	$\frac{1}{\sqrt{10}}[-3, 1]$	$\frac{1}{\sqrt{2}}[-1, 1]$

Layer 1, one step at a time:

$$\mathbf{u}_1 = \frac{1}{\sqrt{10}} \begin{bmatrix} 1 \\ 3 \end{bmatrix}$$

$$\mathbf{v}_1^\mathsf{T} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \end{bmatrix}$$

$$\frac{1}{\sqrt{10}} \cdot \frac{1}{\sqrt{2}} = \frac{1}{\sqrt{20}}$$

$$\begin{bmatrix} 1 \\ 3 \end{bmatrix} \begin{bmatrix} 1 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 1 \\ 3 & 3 \end{bmatrix}$$

$$\mathbf{u}_1 \mathbf{v}_1^\mathsf{T} = \frac{1}{\sqrt{20}} \begin{bmatrix} 1 & 1 \\ 3 & 3 \end{bmatrix}$$

Multiply by σ_1:

$$\sigma_1 \mathbf{u}_1 \mathbf{v}_1^\mathsf{T} = \frac{3\sqrt{5}}{\sqrt{20}} \begin{bmatrix} 1 & 1 \\ 3 & 3 \end{bmatrix}$$

$$\sigma_1 \mathbf{u}_1 \mathbf{v}_1^\mathsf{T} = 1.5 \begin{bmatrix} 1 & 1 \\ 3 & 3 \end{bmatrix}$$

$$\sigma_1 \mathbf{u}_1 \mathbf{v}_1^\mathsf{T} = \begin{bmatrix} 1.5 & 1.5 \\ 4.5 & 4.5 \end{bmatrix}$$

Layer 2:

$$\sigma_2 \mathbf{u}_2 \mathbf{v}_2^\mathsf{T} = \frac{\sqrt{5}}{\sqrt{20}} \begin{bmatrix} 3 & -3 \\ -1 & 1 \end{bmatrix}$$

$$\sigma_2 \mathbf{u}_2 \mathbf{v}_2^\mathsf{T} = 0.5 \begin{bmatrix} 3 & -3 \\ -1 & 1 \end{bmatrix}$$

$$\sigma_2 \mathbf{u}_2 \mathbf{v}_2^\mathsf{T} = \begin{bmatrix} 1.5 & -1.5 \\ -0.5 & 0.5 \end{bmatrix}$$

The sum:

$$\begin{bmatrix} 1.5 & 1.5 \\ 4.5 & 4.5 \end{bmatrix}$$

$$+ \begin{bmatrix} 1.5 & -1.5 \\ -0.5 & 0.5 \end{bmatrix}$$

$$= \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix} = A$$

The first layer already has the right overall size: its bottom row $[4.5, 4.5]$ is close to A's $[4, 5]$. The second layer is a smaller correction.

The formal version. A matrix of **rank** (G-1627) r is the sum of r rank-1 layers (MML §4.6):

$$A = U\Sigma V^\mathsf{T} = \sum_{i=1}^{r} \sigma_i \, \mathbf{u}_i \mathbf{v}_i^\mathsf{T}$$

Here U holds the columns $\mathbf{u}_i$, V the columns $\mathbf{v}_i$, and Σ (capital sigma) is the **diagonal matrix** (G-601) with $\sigma_1, \sigma_2, \ldots$ on its diagonal and 0 elsewhere. The sign $\sum_{i=1}^{r}$ means "add the terms for $i = 1, 2, \ldots, r$". Check: for A, $r = 2$ and the two terms are the two layers just added.

Why each $\mathbf{u}_i$ pairs only with its own $\mathbf{v}_i$: in the product $U\Sigma V^\mathsf{T}$, column i of U meets row j of V^T through the entry of Σ in row i, column j. That entry is 0 unless $i = j$, so all cross terms vanish. Layers with $\sigma_i = 0$ add nothing, so only the first r count.

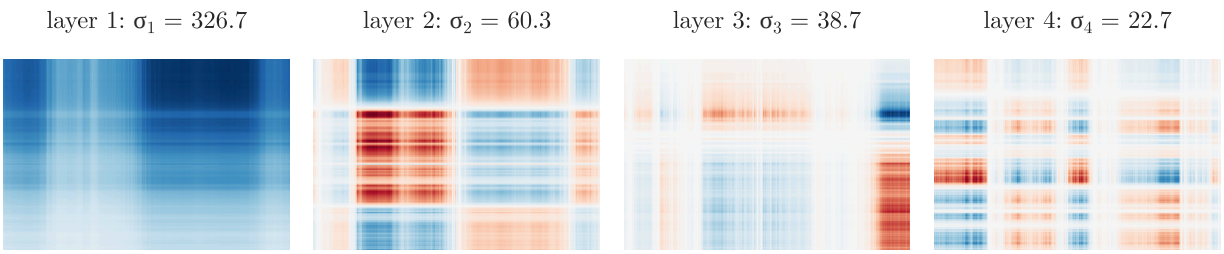


Figure 4: The first four layers $\sigma_i \mathbf{u}_i \mathbf{v}_i^\mathsf{T}$ of the photo: blue positive, red negative, white zero

Figure 4 shows the first four layers of the photo of Figure 1. Each is a column times a row, so each is a pattern of horizontal and vertical stripes, like the plan table. The first one is a blurred brightness map; later ones add finer corrections, positive (blue) in some places and negative (red) in others.

Python: One layer is `s[i] * np.outer(U[:, i], Vt[i])`.

```
import numpy as np

A = np.array([[3, 0], [4, 5]])
U, s, Vt = np.linalg.svd(A)
L1 = s[0] * np.outer(U[:, 0], Vt[0])   # [[1.5, 1.5], [4.5, 4.5]]
L2 = s[1] * np.outer(U[:, 1], Vt[1])   # [[1.5, -1.5], [-0.5, 0.5]]
L1 + L2                                # gives back A
```

NumPy's signs of $\mathbf{u}_i$ and $\mathbf{v}_i$ may both be flipped; their outer product, and so the layer, is the same.

3. The rank k approximation

Key point: Keep the first k layers and drop the rest. The result has **rank** (G-1627) k and needs only $k(m+n+1)$ numbers.

The idea in plain words. Go back to Figure 3 and throw away layer 2, the park correction. What is left is layer 1 alone. Round its entries to whole boxes:

Branch	Mon	Tue	Sat	Sun
small: 1.93, 1.93, 4.07, 6.00 rounded	2	2	4	6
medium: 3.86, 3.86, 8.14, 12.00 rounded	4	4	8	12
large: 6.11, 6.11, 12.89, 19.00 rounded	6	6	13	19

The rounded layer 1 is exactly the real table, park extras included. One layer was enough here because the dropped layer was tiny ($\sigma_2 = 0.33$ against $\sigma_1 = 30.03$).

The standard term. Keeping the k largest **singular values** (G-1812) with their vectors and dropping the rest gives the **rank k approximation** of A, also called the **truncated SVD** (G-1630). It is written $\hat{A}_k$, read "A hat k".

Worked example on the 2 × 2 matrix. For A, the rank-1 approximation is its first layer:

$$\hat{A}_1 = \begin{bmatrix} 1.5 & 1.5 \\ 4.5 & 4.5 \end{bmatrix}$$

The formal version (MML §4.6):

$$\hat{A}_k = \sum_{i=1}^{k} \sigma_i \, \mathbf{u}_i \mathbf{v}_i^{\mathsf{T}} = U_k \Sigma_k V_k^{\mathsf{T}}$$

- U_k: the first k columns of U;
- Σ_k: the top-left $k \times k$ block of Σ;
- V_k: the first k columns of V.

Check: with $k = 1$ the sum has the one term $\sigma_1 \mathbf{u}_1 \mathbf{v}_1^{\mathsf{T}}$, the matrix $\hat{A}_1$ above.

Storage. Each kept layer costs one column $\mathbf{u}_i$ (m numbers), one singular value σ_i (1 number) and one column $\mathbf{v}_i$ (n numbers). For the sales table, $m = 3$ and $n = 4$:

$$\text{one layer} = 3 + 1 + 4 = 8 \text{ numbers}$$

$$\text{full table} = 3 \times 4 = 12 \text{ numbers}$$

In general:

$$\text{numbers stored} = k(m + n + 1)$$

For the photo of Figure 1, $m = 427$, $n = 640$ and $k = 20$:

$$\text{full photo} = 427 \times 640 = 273{,}280$$

$$427 + 640 + 1 = 1{,}068 \text{ per layer}$$

$$\text{stored} = 20 \times 1{,}068 = 21{,}360$$

$$\text{share} = 21{,}360/273{,}280 = 7.8 \text{ percent}$$

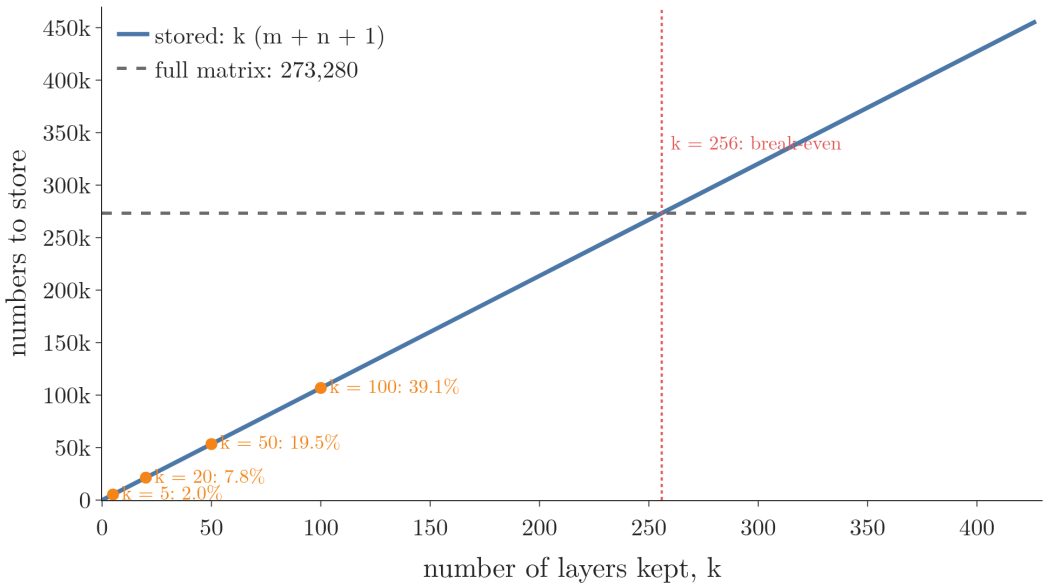


Figure 5: The numbers stored for a rank-k approximation of the 427 × 640 photo, k(m + n + 1), against the 273,280 numbers of the full matrix. k = 20 needs 7.8%; from k = 256 on, the layers cost more than the matrix.

The layered storage only pays off when k is small. Figure 5 shows where the line is crossed for the photo. The blue line climbs by 1,068 numbers per layer; the flat line is the full photo. They meet at

$$256 \times 1{,}068 = 273{,}408 > 273{,}280$$

so from $k = 256$ on, the layers cost more than the matrix itself.

4. The best approximation: Eckart–Young

> **Key point:** No matrix of **rank** (G-1627) k is closer to A than $\hat{A}_k$. Its error, measured as the most the leftover can stretch an arrow, is exactly σ_{k+1}, the first **singular value** (G-1812) left out.

4.1 Measuring the size of a matrix

> **Key point:** The size of a matrix can be measured by the most it can stretch an arrow of length 1. That stretch is its largest **singular value** (G-1812).

The idea in plain words. To say which approximation is "closest", we need to measure how big the leftover (the matrix minus its approximation) is. A vector has a length. A matrix is a machine that turns arrows into arrows (see reading a matrix as a picture). So measure a matrix by feeding it every arrow of length 1 and seeing how long the longest output is.

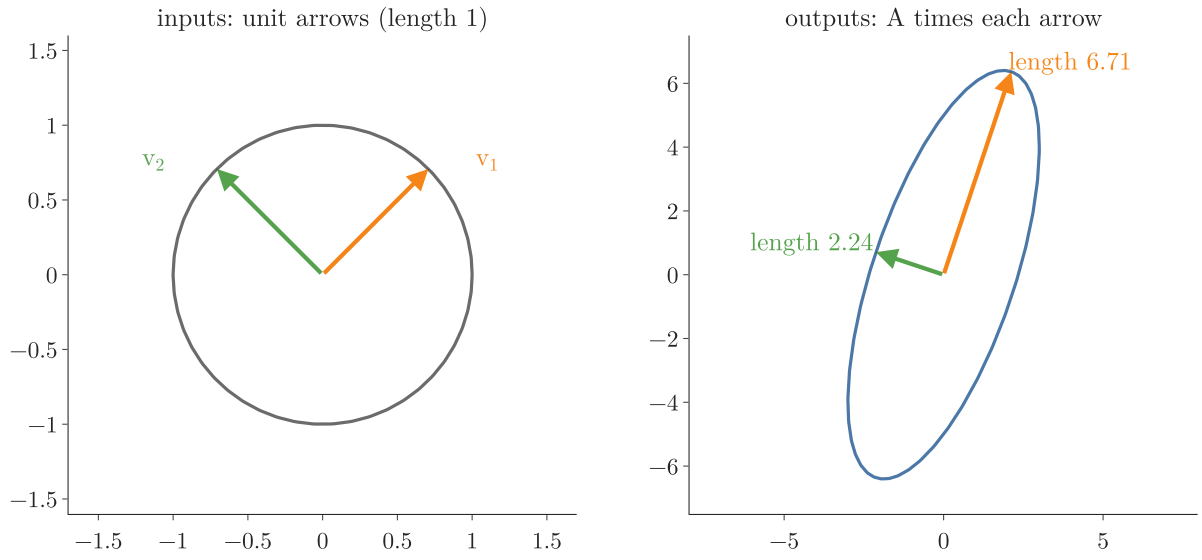


Figure 6: Left: arrows of length 1 end on the unit circle; two of them, v_1 and v_2, are drawn. Right: the matrix A sends the circle to an ellipse; v_1 comes out longest, 6.71, and v_2 shortest, 2.24

Figure 6 does this for A. On the left, the tips of all arrows of length 1 form the unit circle. On the right, A sends the circle to an ellipse. The longest output (orange) has length 6.71; it comes from the input $\mathbf{v}_1$. The shortest (green) has length 2.24, from $\mathbf{v}_2$.

Worked example. Feed A the arrow $\mathbf{v}_1$ of length 1:

$$\mathbf{v}_1 = \frac{1}{\sqrt{2}}[1, 1]$$

$$\mathbf{v}_1 = [0.707, 0.707]$$

Top entry:

$$3 \times 0.707 + 0 \times 0.707 = 2.12$$

Bottom entry:

$$4 \times 0.707 + 5 \times 0.707 = 6.36$$

$$A\mathbf{v}_1 = \begin{bmatrix} 2.12 \\ 6.36 \end{bmatrix}$$

$$\text{length} = \sqrt{2.12^2 + 6.36^2}$$

$$\text{length} = \sqrt{4.5 + 40.5}$$

$$\text{length} = \sqrt{45} = 6.71$$

That is σ_1. By the ellipse picture, no other arrow of length 1 comes out longer.

The standard term and the formula. The largest stretch is the **spectral norm** (G-1852), written $\|M\|_2$ for a matrix M. Here $\mathbf{x}$ is any input arrow and $\|\mathbf{x}\|$ its length (the **L2 norm**, G-1028, the length of an arrow; see magnitude):

$$\|M\|_2 = \max_{\|\mathbf{x}\|=1} \|M\mathbf{x}\| = \sigma_1(M)$$

Read it as: over all arrows $\mathbf{x}$ of length 1, the longest output $M\mathbf{x}$; and that equals the largest singular value of M. Check: $\|A\|_2 = 6.71 = \sigma_1$.

4.2 The Eckart–Young theorem

> **Key point:** Among all **rank** (G-1627) k matrices, the **truncated SVD** (G-1630) leaves the smallest leftover, and the size of that leftover is σ_{k+1}.

The idea in plain words. When we keep the first k layers, the leftover is exactly the layers we dropped. The biggest of those is layer $k+1$, so the leftover can stretch an arrow by σ_{k+1} at most. The surprising part is that no other way of picking a rank k matrix, however clever, leaves a smaller leftover.

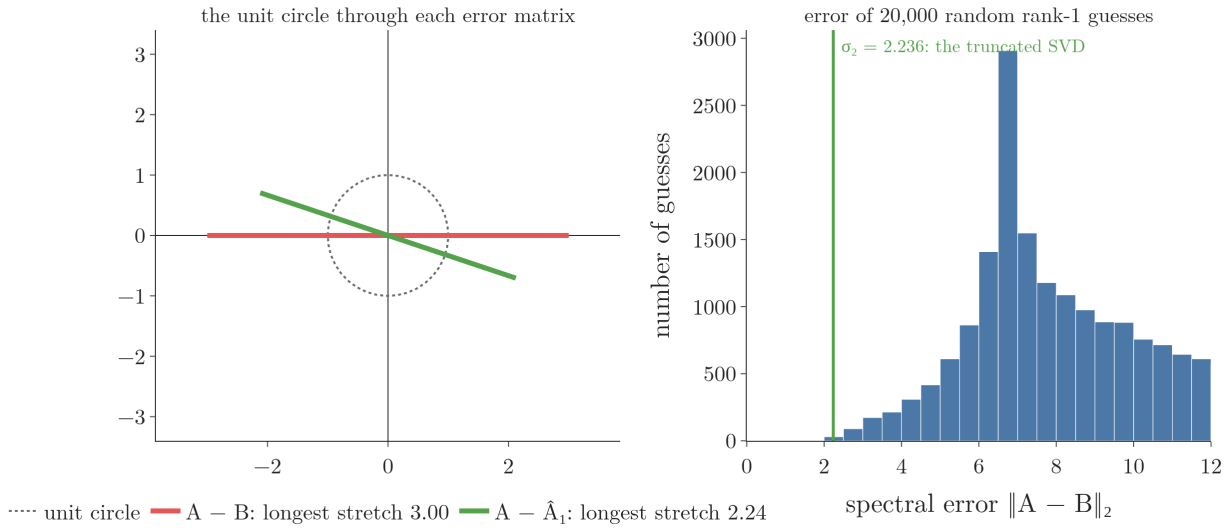


Figure 7: Eckart–Young on the 2×2 example. Left: the unit circle (dotted) sent through the error of the truncated SVD (green, longest stretch 2.24) and through the error of the other guess B (red, longest stretch 3). Right: the spectral errors of 20,000 random rank-1 matrices; none falls below $\sigma_2 = 2.236$ (green line).

Figure 7 tests this on A with $k = 1$. On the left, each leftover is applied to the unit circle. Both leftovers have rank 1, so they flatten the circle onto a line segment; the segment of the truncated SVD's leftover (green) reaches 2.24 from the centre, the one of another guess B (red) reaches 3. On the right, we try 20,000 random rank-1 matrices instead. Their leftovers spread from 2.26 upwards; not one gets below $\sigma_2 = 2.236$.

Worked example. The leftover of the truncated SVD is the dropped layer 2:

$$A - \hat{A}_1 = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix}$$

$$- \begin{bmatrix} 1.5 & 1.5 \\ 4.5 & 4.5 \end{bmatrix}$$

$$A - \hat{A}_1 = \begin{bmatrix} 1.5 & -1.5 \\ -0.5 & 0.5 \end{bmatrix}$$

$$\|A - \hat{A}_1\|_2 = \sigma_2$$

$$\sigma_2 = \sqrt{5} = 2.24$$

A natural other rank-1 guess keeps the second row of A and zeroes the first:

$$B = \begin{bmatrix} 0 & 0 \\ 4 & 5 \end{bmatrix}$$

$$A - B = \begin{bmatrix} 3 & 0 \\ 0 & 0 \end{bmatrix}$$

$$\|A - B\|_2 = 3$$

The leftover of B stretches by 3, more than 2.24.

The standard term and the formula. The **Eckart–Young theorem** (G-657) (MML Thm 4.25; Eckart and Young 1936) says:

$$A - \hat{A}_k = \sum_{i=k+1}^{r} \sigma_i \, \mathbf{u}_i \mathbf{v}_i^{\mathsf{T}}$$

$$\|A - \hat{A}_k\|_2 = \sigma_{k+1}$$

$$\sigma_{k+1} \leq \|A - B\|_2$$

The third line holds for every matrix B of rank k.

Check: for A and $k = 1$, the first line is layer 2, the second gives 2.24, and the third holds for B because $2.24 \leq 3$.

The second line follows from the first. The leftover $A - \hat{A}_k$ is itself written in SVD form, with **singular values** (G-1812) $\sigma_{k+1}, \sigma_{k+2}, \ldots$. Its largest one is σ_{k+1}, and the **spectral norm** (G-1852) of a matrix is its largest singular value. The harder part is the third line: that no other B wins.

> **Extra:** Why no other rank k matrix can win, in outline. Write $\mathbb{R}^n$ for the space of all lists of n numbers: $\mathbb{R}^2$ holds pairs such as $(2, 3)$. A rank k matrix B sends at least an $(n - k)$-dimensional set of inputs to zero; on those inputs $A - B$ acts exactly like A. The first $k + 1$ **right singular vectors** (G-1692) span a $(k + 1)$-dimensional set on which A stretches every vector by at least σ_{k+1}. Two subspaces of $\mathbb{R}^n$ with dimensions adding up to more than n must share a non-zero vector $\mathbf{x}$. On that $\mathbf{x}$, $(A - B)\mathbf{x} = A\mathbf{x}$, which is at least σ_{k+1} times as long as $\mathbf{x}$, so $\|A - B\|_2 \geq \sigma_{k+1}$.

> **Extra:** The theorem also holds for a second common size measure (Eckart and Young 1936), the **Frobenius norm** (G-809), written $\|M\|_F$: the square root of the sum of all squared entries, the L2 norm of the matrix read as one long vector. It also equals the square root of the sum of the squared singular values. For A:
>
> $$\|A\|_F = \sqrt{3^2 + 0^2 + 4^2 + 5^2}$$

$$= \sqrt{9 + 0 + 16 + 25}$$
$$= \sqrt{50}$$
$$\sqrt{\sigma_1^2 + \sigma_2^2} = \sqrt{45 + 5} = \sqrt{50}$$

The truncated SVD's Frobenius error is $\sqrt{\sigma_{k+1}^2 + \sigma_{k+2}^2 + \ldots}$. In NumPy, `np.linalg.norm(M)` is the Frobenius norm and `np.linalg.norm(M, 2)` the spectral norm.

5. Compressing an image

Key point: A grayscale image is a table of brightness values; its **singular values** (G-1812) fall fast, so a few layers carry most of the picture.

A grayscale photo is a matrix: one row per pixel row, one column per pixel column, each entry a brightness between 0 (black) and 1 (white). The photo of Figure 1 is scikit-learn's sample image `china.jpg` turned to grayscale: a 427×640 matrix of **rank** (G-1627) 427. Like the sales table, it has big patterns (a dark lower half, a bright sky) and small details on top.

Reading Figure 1 in order of k:

- $k = 1$ (0.4 percent of the numbers): one column times one row. All we see is a brightness map made of stripes, with the dark lower half and the bright sky.
- $k = 5$ (2.0 percent): the building appears as a dark block.
- $k = 20$ (7.8 percent): the building and the trees are recognisable.
- $k = 50$ (19.5 percent) and $k = 100$ (39.1 percent): close to the original, with some graininess in the flat sky.

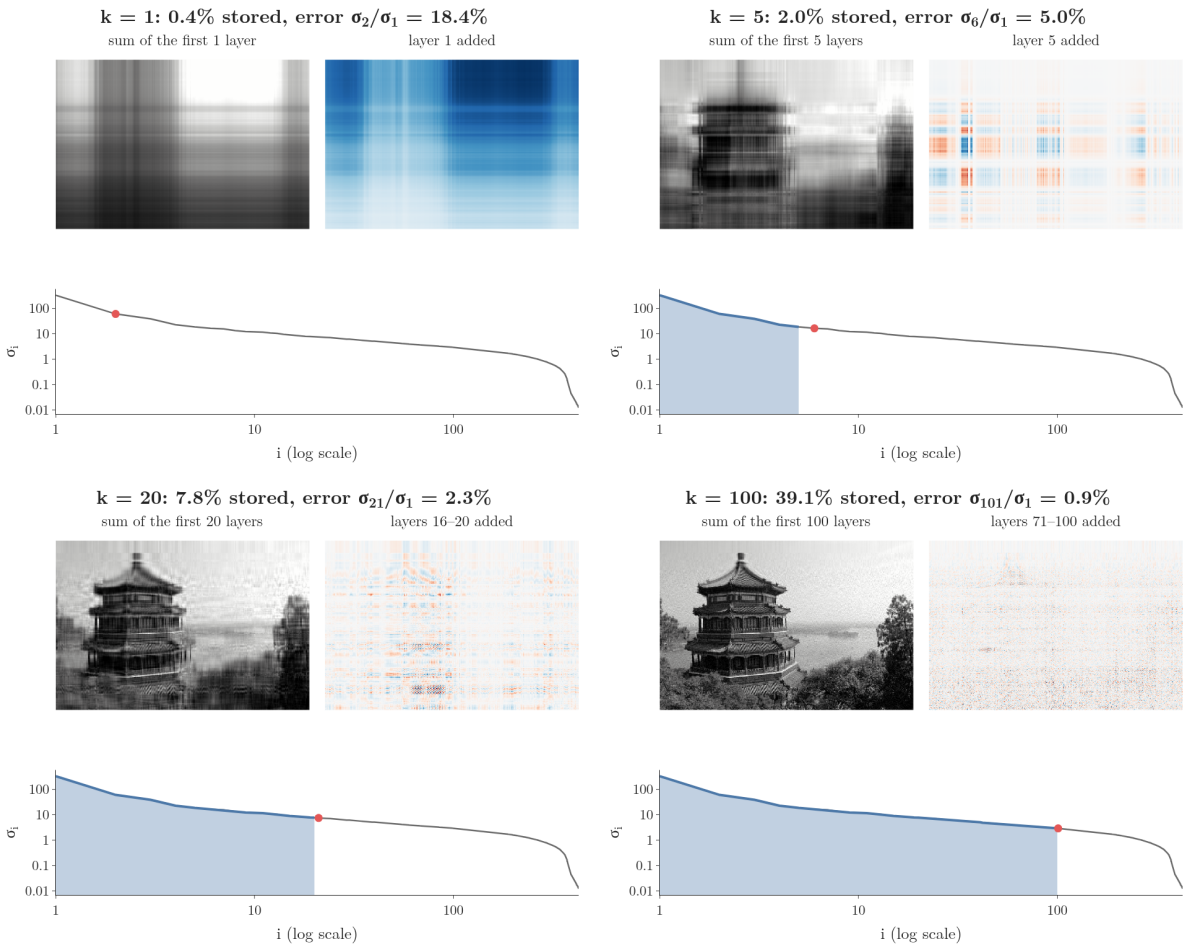


Figure 8: The photo rebuilt layer by layer, k = 1 to 100: the sum of the first k layers (left), the layers just added (right, blue positive, red negative) and the singular values kept so far (filled)

Figure 8 replays the rebuild one layer at a time. Watch the right panel: the first layers are broad stripes that set the overall light and dark, later ones are fine detail, and the red dot on the singular values slides down as each added layer matters less.

The "error" in each title is the spectral error of Section 4, relative to the size of the image:

$$\text{error} = \frac{\sigma_{k+1}}{\sigma_1}$$

k	error
1	18.4 percent
20	2.3 percent

Extra: Real image formats such as JPEG do not use the SVD. They use a fixed set of patterns (cosine waves on small 8×8 blocks), which is faster and needs no U and V to be stored for each picture (Wallace 1991). The SVD example shows the idea of keeping the important directions, not how photos are actually compressed.

6. How many singular values to keep

Key point: Plot the **singular values** (G-1812): they usually fall steeply and then flatten. Keep the steep part, or keep enough to reach a chosen share of the sum of squared singular values.

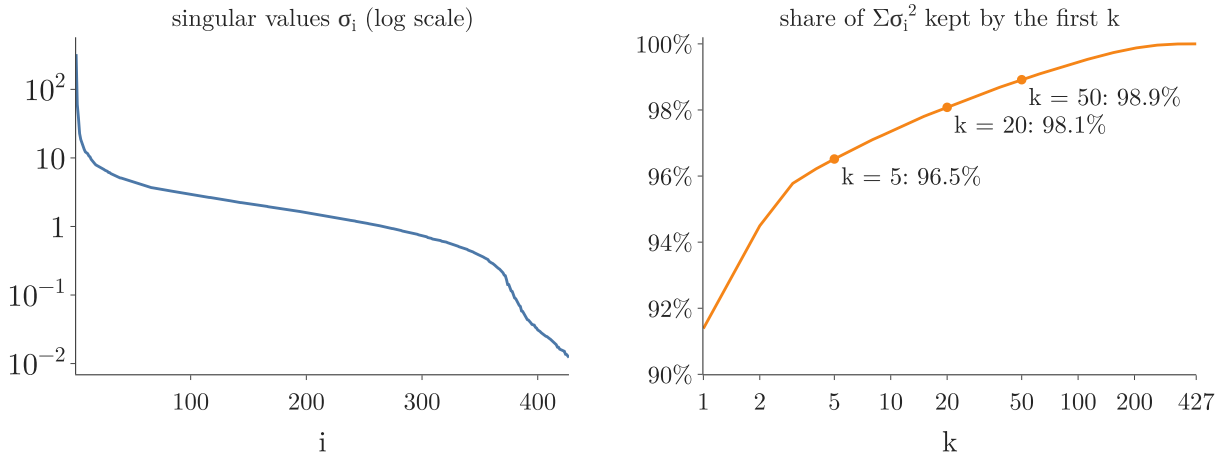


Figure 9: Singular values of the photo, and the share of $\sum \sigma_i^2$ kept by the first k

Figure 9 (left) plots all 427 singular values of the photo on a log scale, where each step up the axis multiplies by 10:

i	σ_i
1	326.7
5	18.6
50	about 4.4
427	about 0.01

A few directions carry most of the picture. Two common ways to choose k:

- **The elbow.** Keep the singular values before the curve flattens: the steep part is structure, the flat part detail or noise.
- **A share of the total.** The sum of the squared singular values equals the sum of all squared entries of the matrix (the **Frobenius norm** (G-809) squared, Section 4.2). Keep enough layers to reach a chosen share of it.

For the sales table the share is easy to read off:

$$\text{share} = \frac{\sigma_1^2}{\sigma_1^2 + \sigma_2^2}$$

$$\text{share} = \frac{30.03^2}{30.03^2 + 0.33^2}$$

$$\text{share} = 99.99 \text{ percent}$$

For the photo, Figure 9 (right) shows the share kept by the first k:

k	share kept
5	96.5 percent
20	98.1 percent
50	98.9 percent

The share rule is the same rule as choosing the number of principal components by explained variance in how many components to keep. The match is not a coincidence: **PCA through the SVD** (G-1468) shows that PCA is the SVD of the centred data matrix (each column minus its mean).

> **Python:** The **rank** (G-1627) k approximation of the photo.
> ```
> from sklearn.datasets import load_sample_image
>
> rgb = load_sample_image("china.jpg")
> img = rgb.astype(float) @ [0.299, 0.587, 0.114] / 255
> U, s, Vt = np.linalg.svd(img, full_matrices=False)
> k = 20
> img_k = (U[:, :k] * s[:k]) @ Vt[:k]
> s[k] / s[0]                                  # 0.023
> ```
> `U[:, :k] * s[:k]` scales column i of U_k by σ_i, which is $U_k\Sigma_k$ without building the **diagonal matrix** (G-601). The grayscale weights 0.299, 0.587 and 0.114 are the standard ones for red, green and blue (ITU-R BT.601). The Notebook has a slider that moves through the ranks.

7. Removing noise

> **Key point:** Random noise spreads thinly over all **singular values** (G-1812), while structure piles up in a few large ones. Keeping only the large ones removes most of the noise.

The idea in plain words. Picture a clean drawing covered in random speckles. The drawing is made of a few big patterns, so it lives in a few big layers. The speckles have no

pattern at all, so they spread a little into every layer. Keeping only the big layers keeps the drawing and throws away most of the speckles.

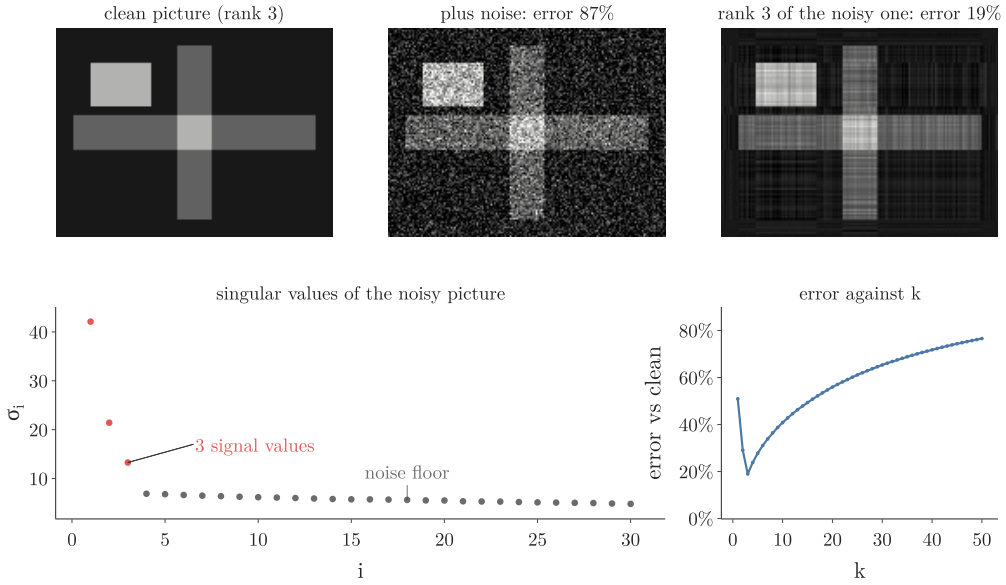


Figure 10: Noise reduction: a rank-3 picture, the same with noise, and the rank-3 approximation of the noisy one

Figure 10 starts from a simple 120×160 picture made of a square and two bars (top left). Each shape is a rectangle of constant brightness, which is an **outer product** (G-1418) (a column of 1s and 0s times a row of 1s and 0s), so the picture has rank 3. We add random noise to every pixel, normal with standard deviation 0.3 (top middle).

The singular values of the noisy picture (Figure 10, bottom left) split into two groups:

- three large ones, 42.1, 21.4 and 13.3: the shapes;
- a flat **noise floor** (G-1328) of values near 5 to 7: the noise.

The noise has no structure, so no direction is special and its contribution spreads over all singular values at a low level. The shapes are concentrated in three directions, so they stand out above the floor. The height of the floor can be predicted: for an $m \times n$ matrix of pure noise with standard deviation s, the largest singular value is close to $s(\sqrt{m} + \sqrt{n})$ (Gavish and Donoho 2014). Here:

$$\sqrt{120} + \sqrt{160} = 10.95 + 12.65 = 23.6$$

$$0.3 \times 23.6 = 7.1$$

the top of the observed floor.

Worked numbers. We measure the error as the size of the difference from the clean picture, relative to the size of the clean picture:

Picture	error
noisy picture itself	87 percent
rank-3 approximation	19 percent
rank-4 approximation	24 percent
rank-10 approximation	41 percent
rank-50 approximation	77 percent

Keeping more than 3 layers makes it worse again (Figure 10, bottom right), because each extra layer adds back mostly noise.

The formula. With A_{clean} the clean picture and both sizes in the **Frobenius norm** (G-809):

$$\text{error} = \frac{\|\hat{A}_k - A_{\text{clean}}\|_F}{\|A_{\text{clean}}\|_F}$$

The same idea, keeping only the singular values above the noise floor, is a standard way to remove noise from a data matrix (Gavish and Donoho 2014). Picking k at the edge of the noise floor is the elbow rule of Section 6.

8. Summary

Idea	Formula	Example	Why it matters
Outer product	$\mathbf{u}\mathbf{v}^\mathsf{T}$, entry $u_i v_j$; rank 1	sizes $[1,2,3]$ times days $[2,2,4,6]$: 12 entries from 7 numbers	a whole table from a few numbers
Layers	$A = \sum_i \sigma_i \mathbf{u}_i \mathbf{v}_i^\mathsf{T}$	sales: $\sigma_1 = 30.03$, $\sigma_2 = 0.33$	the big patterns come first
Rank k approximation	$\hat{A}_k = U_k \Sigma_k V_k^\mathsf{T}$	$\hat{A}_1$ of A has rows $[1.5, 1.5]$, $[4.5, 4.5]$	keeps the main pattern, drops the small corrections
Storage	$k(m+n+1)$	photo, $k = 20$: 21,360 of 273,280 (7.8 percent)	pays off only for small k: from $k = 256$ the photo's layers cost more than the photo
Spectral norm	$\|M\|_2 = \sigma_1(M)$	$\|A\|_2 = 6.71$	measures a matrix by the most it can stretch a unit arrow
Eckart–Young	$\|A - \hat{A}_k\|_2 = \sigma_{k+1}$, the smallest possible	error of $\hat{A}_1$: 2.24; another rank-1 guess: 3	no rank k matrix does better
Noise reduction	keep the σ_i above the noise floor	error 87 percent to 19 percent at $k = 3$	structure sits in a few large σ_i, noise spreads thinly over all

- Every matrix is a weighted sum of **rank-1 layers** (G-1631), largest weight first, so the first layers carry the main pattern.
- Dropping the small layers gives the best **rank** (G-1627) k approximation, with error σ_{k+1}, so the first **singular value** (G-1812) left out says exactly how much is lost.
- Fast-falling singular values mean a matrix can be compressed well, because a few layers then carry most of it.
- Noise forms a flat floor of small singular values; truncating below it removes most of the noise, because noise has no special direction while the structure piles up in a few large layers.
- So keeping only the big layers of the SVD gives the best small summary of a table, whether the goal is to compress it or to clean it.

9. Sources

Built from

- Deisenroth, M. P., Faisal, A. A. and Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. Section 4.6 and Theorem 4.25 (MML).

Other references

- Eckart, C. and Young, G. (1936). "The approximation of one matrix by another of lower rank". *Psychometrika* 1(3).
- Gavish, M. and Donoho, D. L. (2014). "The Optimal Hard Threshold for Singular Values is $4/\sqrt{3}$". *IEEE Transactions on Information Theory* 60(8).
- ITU-R Recommendation BT.601. *Studio encoding parameters of digital television.* Luma weights 0.299, 0.587, 0.114.
- Wallace, G. K. (1991). "The JPEG Still Picture Compression Standard". *Communications of the ACM* 34(4).

10. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Eckart–Young theorem (G-657)	The theorem that the truncated SVD (keeping the k largest singular values) is the closest rank k matrix to A, so it is the best way to compress A to rank k; its spectral error is σ_{k+1}.
Rank-1 layer (G-1631)	One piece of a matrix split by the SVD: a column times a row, $\sigma_i \mathbf{u}_i \mathbf{v}_i^{\mathsf{T}}$, weighted by its singular value σ_i; adding all the layers gives back the matrix.
Rank k approximation (truncated SVD) (G-1630)	The approximation of a matrix that keeps only its k largest singular values with their vectors and drops the rest, $\hat{A}_k = U_k \Sigma_k V_k^{\mathsf{T}}$; it is the best small summary of the matrix and needs far fewer numbers.
Spectral norm (G-1852)	The largest stretch a matrix gives to any input of length 1, $\|M\|_2 = \sigma_1$; it is one way to measure the size of a matrix.
Frobenius norm (G-809)	One number for the size of a matrix: the square root of the sum of all its squared entries, also equal to $\sqrt{\sum \sigma_i^2}$ over its singular values σ_i.

Term	Meaning
Noise floor (G-1328)	The flat run of small singular values that random noise produces; the few singular values standing above it carry the real structure, so keeping only those removes most of the noise.
Rank (of a matrix) (G-1627)	The number of linearly independent columns of a matrix, that is, how many directions its columns really span; a column that is a combination of others does not add to it.
Singular value (σ_i) (G-1812)	A stretch factor of a matrix: the length of $A\mathbf{v}_i$; never negative, listed largest first.
Outer product (G-1418)	Multiplying a column vector by a row vector to get a whole matrix: $\boldsymbol{\delta}\boldsymbol{\delta}^{\mathsf{T}}$ has entry $\delta_i\delta_j$ for every pair; it is what $\boldsymbol{\delta}^2$ means in a Taylor polynomial of several variables.
Transpose (G-2012)	A matrix or vector with rows and columns swapped; it turns a column vector u into a row u^{T}, so $u^{\mathsf{T}}x$ is the dot product.
Span (G-1838)	The set of all linear combinations of some vectors: every point reachable by scaling and adding them, such as the whole plane for $\hat{\imath}$ and $\hat{\jmath}$.
Left singular vector ($\mathbf{u}_i$) (G-1079)	One of the output directions of the SVD: the unit vector that a right singular vector lands on, stretched by its singular value ($A\mathbf{v}_i = \sigma_i\mathbf{u}_i$); the columns of U.
Right singular vector ($\mathbf{v}_i$) (G-1692)	A column $\mathbf{v}_i$ of V in the SVD: a unit-length input direction that the matrix sends to σ_i times the matching left singular vector $\mathbf{u}_i$.
Diagonal matrix (G-601)	A matrix with zeros everywhere off the diagonal; it scales each axis by its own factor.
L2 norm (norm, magnitude, length of a vector) (G-1028)	The usual magnitude of a vector, its distance from the origin: the square root of the sum of squared components, $\|w\| = \sqrt{w_1^2 + w_2^2 + \ldots}$.
PCA through the SVD (G-1468)	Computing PCA from the SVD of the centred data: the columns of V are the principal components, σ_i^2/n their variances, and $U\Sigma$ the data's new coordinates (scores).